

Resumo Design by Contract

O artigo parte de uma observação sobre a literatura de programação orientada a objetos. Apesar dos avanços em modularidade e reutilização, pouco foi dito sobre confiabilidade. Para o autor, isso é grave porque componentes reutilizados em vários contextos precisam ter seu comportamento especificado com clareza, e não há como fazer isso sem algum mecanismo formal. O artigo propõe uma abordagem baseada numa metáfora simples, a construção de software como uma sucessão de decisões contratuais documentadas.

A proposta é tratar a relação entre uma rotina e quem chama de contrato. O cliente cumpre as pré-condições, a rotina garante as pós-condições, e a classe mantém suas invariantes. Se a pré-condição não foi satisfeita, a culpa é de quem chamou. Se a pós-condição foi violada, a culpa é da implementação. Essa clareza na atribuição de responsabilidades é o que o autor defende como o principal valor da abordagem. No Eiffel, a linguagem usada como base ao longo do texto, esses elementos aparecem diretamente na sintaxe das rotinas, com as cláusulas `require` e `ensure` delimitando as obrigações de cada parte. Uma pré-condição ausente equivale a `require true`, ou seja, qualquer chamada é válida. Uma pós-condição ausente equivale a `ensure true`, ou seja, nada é garantido.

As restrições de classe complementam as pré e pós-condições ao expressar propriedades que devem valer para todas as instâncias em qualquer estado observável, isto é, após a criação de um objeto e após a execução de qualquer rotina exportada. Uma restrição não precisa valer durante a execução interna de uma rotina, apenas nos estados visíveis ao cliente. Isso significa que ela é implicitamente acrescentada tanto à pré-condição quanto à pós-condição de toda rotina exportada, funcionando como uma cláusula geral que se aplica a todos os contratos da classe.

O artigo critica a programação defensiva. Verificar as mesmas condições em vários lugares ao mesmo tempo parece prudente, mas o resultado é código cheio de ruído onde fica difícil distinguir o processamento útil da gestão de erros. O contrato resolve isso ao dizer explicitamente quem é responsável por quê, tornando as verificações redundantes desnecessárias. Meyer argumenta que a robustez de um sistema não depende de cada rotina tentar cobrir todos os casos possíveis, mas da coerência das interfaces e da simplicidade de cada módulo. Rotinas com pré-condições fortes fazem um trabalho bem definido e fazem bem, em vez de tentar lidar com todas as situações imagináveis.

O artigo também discute exceções, com críticas à abordagem de linguagens como Ada e CLU. O problema central é que, sem contratos, não há uma definição clara do que é um caso excepcional, e o mecanismo de exceções acaba sendo usado para qualquer coisa, inclusive para situações que seriam melhor tratadas com estruturas de controle normais. Meyer analisa exemplos concretos de código Ada onde exceções são

levantadas para cobrir pré-condições não formuladas, como chamar uma operação de remoção em uma pilha vazia, e mostra que em todos esses casos a solução correta seria definir a pré-condição explicitamente e deixar a responsabilidade de respeitá-la com o cliente.

A proposta do Eiffel é mais restrita. Quando uma rotina não consegue cumprir seu contrato, ela deve tentar novamente com uma estratégia alternativa ou devolver o sistema a um estado consistente e sinalizar falha ao chamador. Não existe a opção de retornar silenciosamente como se tudo tivesse dado certo. Isso é expresso no Eiffel através das cláusulas `rescue` e `retry`. A cláusula `rescue` é acionada quando ocorre uma exceção durante a execução da rotina e pode conter uma instrução `retry`, que reinicia o corpo da rotina com uma estratégia alternativa. Se a cláusula `rescue` terminar sem `retry`, a rotina falha e sinaliza isso ao chamador através de uma nova exceção. Em ambos os casos, a rotina deve antes restaurar as restrições da classe, garantindo que o objeto permaneça em um estado consistente mesmo após a falha.

A última parte do artigo conecta os contratos à herança. Quando uma subclasse redefine uma rotina, ela está subcontratando, e um subcontratante honesto não pode exigir mais do cliente nem entregar menos. Isso se traduz em uma regra concreta. Pré-condições de rotinas redefinidas só podem ser afrouxadas, e pós-condições só podem ser fortalecidas. No Eiffel, isso é reforçado pela linguagem através das construções `require else` e `ensure then`, que combinam as novas asserções com as originais usando operadores lógicos que garantem automaticamente o enfraquecimento da pré-condição e o fortalecimento da pós-condição. Essa restrição é o que dá à herança um significado semântico preciso. Sem ela, uma subclasse poderia redefinir uma rotina de forma arbitrária, violando as expectativas de qualquer cliente que opere sobre o tipo original. Com ela, a redefinição se torna uma transformação que preserva a semântica, e não apenas um mecanismo de reuso de código.

O valor do texto está em mostrar que problemas como exceções mal usadas e verificações defensivas em cascata não são acidentes. São sintomas da falta de um vocabulário para falar sobre o que uma rotina deve e não deve fazer. Os contratos oferecem esse vocabulário, e o artigo demonstra como esse vocabulário se aplica de forma consistente a situações que a engenharia de software costuma tratar de maneira improvisada.